EC327 Midterm 1 Cheat Packet v2

Dense but scan-friendly handbook focused on casting, syntax repair, function signatures, arrays, file I/O, dynamic memory, and pointer diagrams. Built from your notes, practice code, and the topics covered before the midterm.

1. Exam survival	Compile commands, common syntax failures, quick workflow
2. Casting and arithmetic	static_cast, integer division, narrowing vs widening, mixed expressions
3. Control flow	if / switch / while / for and semicolon traps
4. Functions and headers	prototype matching, separate compilation, value/reference/address
5. Arrays and file I/O	array traversal, arrays of pointers, course-style file reading patterns
6. Dynamic memory + pointers	new/delete/delete[], pointer diagrams, pointerSwap, leaks/dangling
7. Example bank	compact examples you can copy under time pressure

How to use this packet

Scan the blue section headings first. Inside each page, the light boxes are the copyable patterns.

When a prompt gives you a function call or four prototype choices, let the call style tell you whether the function wants a value, reference, or pointer.

For this course, some fill-in questions may expect course-style patterns even if a more robust C++ pattern also exists. This packet labels those cases explicitly.

Top 4 stress mistakes from the practice run

Forgetting a cast in arithmetic, so `1/3` becomes `0` before multiplying.

Confusing reference syntax with address-of syntax because both use &.

Mixing up `delete` and `delete[]`.

Changing a local pointer copy instead of changing the caller's pointer. That requires `int *&ptr`.

1. Exam survival

Start here before debugging logic.

Task	Pattern
Compile one file	g++ file.cpp
Compile and name output	g++ -o prog file.cpp
Compile multiple files	g++ -o prog main.cpp helper.cpp other.cpp
Run	./prog
Header guards	#ifndef NAME / #define NAME / ... / #endif

Quick syntax survival checklist

- Missing semicolon after a declaration or statement.
- Single quotes vs double quotes: `'Y'` is a char, `"Yes"` is a string literal.
- Wrong stream operators: input uses >>; output uses <<.
- Misspelled types or names: `double`, not `doble`.
- Prototype and definition must match in return type, parameter types, and parameter order.
- If the prompt says do not make major changes, try to fix the function/interface to match the given call site instead of rewriting the call.

```
// Very common patterns
#include <iostream>
using namespace std;

int main()
{
    int x;
    cin >> x;
    cout << "x = " << x << endl;
    return 0;
}
```

When something compiles but feels wrong

Check whether you silently used integer division.

Check whether a function was supposed to change the caller's variable but you passed by value.

Check whether a pointer points to one object or to a heap array. That determines `delete` vs `delete[]`.

2. Casting and arithmetic - the must-remember page

Casting was one of your weak spots, so this page is intentionally concrete.

Idea	Example	Result / meaning
Implicit narrowing	int a = 1.7;	a becomes 1
C-style cast	int a = (int)1.7;	works, but less explicit
C++ cast	int a = static_cast<int>(1.7);	preferred for clarity
Implicit widening	double d = 4;	d becomes 4.0
Mixed arithmetic	1 / 2	0 because both operands are int
Mixed arithmetic with cast	static_cast<double>(1) / 2	0.5

```
double i = 1.7;
int defaultCast = i;                 // 1
int cStyleCast = (int)i;             // 1
int staticCastInt = static_cast<int>(i); // 1

int j = 4;
double asDouble = static_cast<double>(j); // 4.0

cout << 1 / 2 << endl;                    // 0
cout << static_cast<double>(1) / 2 << endl; // 0.5
```

Fast rules

If both sides of `/` are integers, the result is integer division.

Put the cast before the division if you need a real-number answer.

Use `static_cast<type>(expr)` when the cast is intentional and you want the grader to see it clearly.

2. Casting and arithmetic - common exam patterns

Pattern 1 - prevent integer division from killing your formula

```
// Wrong if you want one third of sum:
score = ave + (1/3 * sum);    // 1/3 becomes 0

// Better:
score = ave + (1.0/3.0) * sum;
// or
score = ave + sum / 3.0;
// or
score = ave + static_cast<double>(sum) / 3;
```

Pattern 2 - convert minutes to hours explicitly

```
double hours = minutes / 60.0;    // good
double mph = MARATHON / hours;

// If time is already in hours:
double mph = MARATHON / bestTime;
```

Pattern 3 - control the final type

```
double ave = 3.6;
int sum = 8;

double raw = ave + sum / 3.0;       // raw is double
int score = static_cast<int>(raw);  // explicit cut to int
```

Expression	Why it matters
(a + b) / 2	If a and b are int, result is int unless you cast.
(a + b) / 2.0	Forces floating-point average.
static_cast<double>(count)	Useful when count is int but formula should stay real-valued.
static_cast<int>(d)	Drops the fractional part.
static_cast<char>(n)	Converts an int to a character code.

Exam habit

Whenever a numeric answer looks suspiciously small, large, or zero, inspect every division and ask: were both operands integers?

If a formula combines int and double values, consider whether the final answer should be kept as double or cast back to int on purpose.

3. Control flow and logic traps

if / else

```
if (answer == 'Y')
{
    date += 1;
}
else
{
    date -= 1;
}
```

switch

```
switch(choice)
{
    case 0:
        cout << "mph";
        break;
    case 1:
        cout << "min/mile";
        break;
    default:
        cout << "error";
}
```

while

```
while (i < n)
{
    sum += list[i];
    i++;
}
```

for

```
for (int i = 0; i < n; i++)
{
    sum += list[i];
}
```

Semicolon trap

```
for (int i = 0; i < n; i++);   // BAD: empty loop body
{
    cout << i;                   // now this block is separate
}
```

Trap	Fix
Forgot break in switch	Execution falls through into the next case.
Used = instead of ==	Comparison in if should usually use ==.
Loop never changes variable	Update the loop variable or condition.
Extra semicolon after if/for/while	Delete it unless you intentionally want an empty statement.

4. Functions, prototypes, headers, and separate compilation

Three separate ideas

```
// Prototype / declaration
int max(int a, int b);

// Definition
int max(int a, int b)
{
    return (a > b) ? a : b;
}

// Call / invoke
int best = max(x, y);
```

Header + implementation split

```
// myTools.h
#ifndef MYTOOLS_H
#define MYTOOLS_H
double calcSplit(const double bestTime, const int metric, const char units);
#endif

// myTools.cpp
#include "myTools.h"
double calcSplit(const double bestTime, const int metric, const char units)
{
    ...
}
```

Prototype matching rule

To fill in a prototype, match the return type, then each parameter's type, then whether the function must modify a caller variable.

If the main file calls `f(x)`, that suggests value or reference. If it calls `f(&x)`, that suggests pointer.

If the prompt says do not make major changes, prefer adapting the function signature to the given code rather than rewriting the call site.

```
// Multi-file compile
g++ -o q1 Q1.cpp timeCalc.cpp raceCalc.cpp
g++ -o q2 Q2.cpp gpaTools.cpp futureCalc.cpp
g++ -o q3 Q3.cpp
```

4. Pass by value vs reference vs address

This is the page to re-read before any prototype question.

Style	Function header	Call	Can it change caller variable?
Value	void f(int x)	f(a)	No
Reference	void f(int &x)	f(a)	Yes
Address / pointer	void f(int *x)	f(&a)	Yes, through *x
Reference to pointer	void f(int *&ptr)	f(p)	Yes, can rebind caller's pointer

Reference

```
void change(int &x)
{
    x = 10;
}

int a = 5;
change(a);    // a becomes 10
```

Address / pointer

```
void change(int *x)
{
    *x = 10;
}

int a = 5;
change(&a);   // a becomes 10
```

Why both use &

```
int &ref = a;   // declaration: ref is a reference
change(&a);      // expression: take the address of a
```

Decision rule

Use reference when you want cleaner syntax and the caller always has a normal variable.

Use pointer when the course/problem is already in pointer style, or you need to pass an address explicitly.

Use reference to pointer when the function must make the caller's pointer point somewhere new.

5. Arrays: creation, traversal, and functions

Create and traverse

```
const int SIZE = 4;
int list[SIZE] = {1, 2, 3, 4};

for (int i = 0; i < SIZE; i++)
{
    cout << list[i] << endl;
}
```

Pass array to a function

```
void printList(const int list[], const int size)
{
    for (int i = 0; i < size; i++)
        cout << list[i] << endl;
}
```

Pattern	Meaning
int list[10];	array of 10 ints on the stack
list[i]	element i
list	in most expressions, acts like a pointer to the first element
list[i] == *(list + i)	array indexing and pointer arithmetic match

```
// Copy or count through an array
int countMatch(const int list[], const int size, const int target)
{
    int count = 0;
    for (int i = 0; i < size; i++)
        if (list[i] == target)
            count++;
    return count;
}
```

Fast reminders

Always pass the size along with the array unless the size is known some other way.

Use `const` on array parameters if the function should not modify the array.

Off-by-one bug check: last valid index is `size - 1`.

5. Arrays: array of ints vs array of pointers vs heap array

Code	What it creates
int array[5];	array of 5 actual ints
int *array[5];	array of 5 pointers to int
int *array = new int[5];	pointer to the first element of a heap array of 5 ints

```
int a = 10, b = 20;
int *ptrArray[2];
ptrArray[0] = &a;
ptrArray[1] = &b;

cout << *ptrArray[0] << endl;   // 10
cout << *ptrArray[1] << endl;   // 20
```

This is the exact question you asked:

`int* array[size]` creates an array whose elements are pointers. Each slot stores an address, not an int value itself.

Heap array pattern

```
int size = 5;
int *heapList = new int[size];

for (int i = 0; i < size; i++)
    heapList[i] = 0;

// later
delete [] heapList;
```

Do not mix these up

`int *p = new int;` means one heap int. Use `delete p;`

`int *p = new int[n];` means a heap array. Use `delete[] p;`

`int *arr[5];` is not a heap array. It is a stack array that stores 5 pointers.

5. File I/O: course-style and general-style patterns

Course fill-in pattern (use when the prompt makes you choose among provided options)

```
ifstream input;
input.open(inputFile);

int sum = 0;
int temp;

while (!input.eof())   // course-style fill-in pattern
{
    input >> temp;
    sum += temp;
}

input.close();
```

Safer general C++ pattern

```
ifstream input;
input.open(inputFile);

int sum = 0;
int temp;
while (input >> temp)
{
    sum += temp;
}

input.close();
```

Read into an array

```
void ReadData(const char *filename,
              int &size, int myArray[])
{
    ifstream inFile;
    inFile.open(filename);
    size = 0;
    int value;
    while (inFile >> value)
    {
        myArray[size] = value;
        size++;
    }
    inFile.close();
}
```

Write to a file

```
ofstream outFile;
outFile.open(filename);

for (int i = 0; i < N; i++)
    outFile << data[i] << endl;

outFile.close();
```

Fast checklist

Use `ifstream` for input and `ofstream` for output.

The filename parameter is commonly `const char *filename` in this course.

If the prompt gives you four exact choices, choose from those choices even if you know a stronger pattern outside the list.

6. Dynamic memory: new, delete, delete[]

Code	Meaning	Cleanup
int *p = new int;	one heap int	delete p;
int *p = new int[10];	heap array of 10 ints	delete [] p;
double *values = new double[40];	heap array of doubles	delete [] values;
short *q = new short;	one heap short	delete q;

```
int *p = new int;
*p = 33;
delete p;
p = NULL;   // optional safety habit in this course
```

Never do this

```
int *p = new int[0];
p[0] = 50;      // invalid for the purpose of storing an element
```

Grow an array

```
double *addNewClass(const double *const oldList,
                    const double newGpa, int &size)
{
    double *returnList = new double[size + 1];
    for (int i = 0; i < size; i++)
        returnList[i] = oldList[i];
    returnList[size] = newGpa;
    size++;
    return returnList;
}
```

Typical bugs

Using `delete` on a heap array allocated with `new[]`.

Reading memory after it has been deleted.

Allocating local/stack memory when the prompt needs something that survives after the function ends.

6. Pointer diagrams: how to translate picture -> code

Workflow

Step	What to write in code
1. Create ordinary variables	Example: `int i = 10;`
2. Create pointer variables	Example: `int *p;`
3. Bind pointers to addresses	Example: `p = &i;`
4. Create heap objects if the picture has anonymous boxes	Example: if the picture has a box with no name, `p = new int;`
5. Fill arrays or pointer arrays slot by slot	Example: `ptrArray[0] = &a;`
6. Only delete memory that was created with `new` - and match the allocation form	Use `delete` for `new int;` and `delete[]` for `new int[40];`

Legend

```
int x = 5;       // one int variable
int *p = &x;     // p stores the address of x
*p = 9;          // change the value stored at the address
p = new int;     // p now points to a heap int
*p = 33;         // fill that heap int with 33
```

Exam habit

Look carefully at whether two arrows point to the same object. That means aliasing.

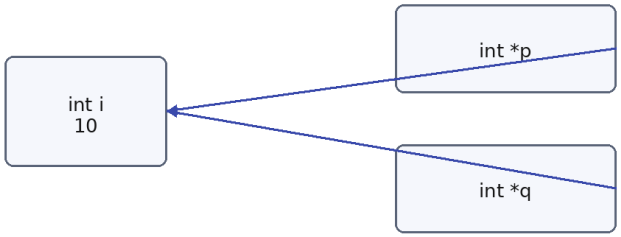
If the box in the picture has no variable name, it is often a heap object created with `new`.

If the picture shows one variable changing where a pointer points, that may require `int *&ptr` in the function header.

6. Pointer diagrams: examples A and B

A. Two pointers to one stack variable

Diagram A - aliasing: two pointers to one stack variable



Code

```
int i = 10;
int *p = &i;
int *q = &i;
```

B. Rebinding the caller's pointer to a new heap int

Diagram B - heap int and pointer rebinding

Code

```
void pointerSwap(int *&ptr)
{
    ptr = new int;
    *ptr = 33;
}
```

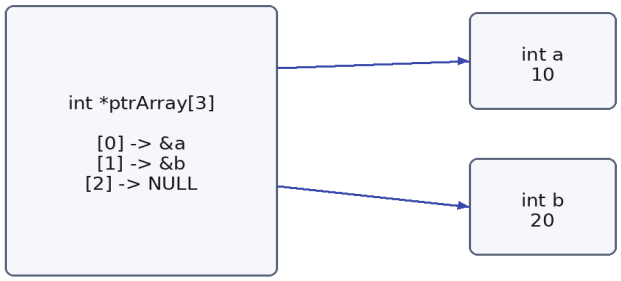
& in int *&ptr means:

ptr is a reference to the caller's pointer,
so rebinding ptr changes the original pointer.

6. Pointer diagrams: examples C and D

C. Array of pointers

Diagram C - array of pointers

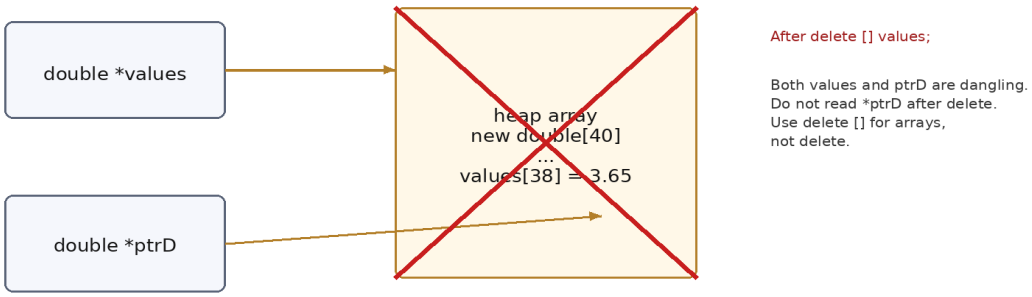


Code

```
int a = 10, b = 20;
int *ptrArray[3];
ptrArray[0] = &a;
ptrArray[1] = &b;
ptrArray[2] = NULL;
```

D. delete vs delete[] and dangling pointers

Diagram D - delete vs delete[] and dangling pointers



6. Pointer bug checklist + const pointer forms

Bug checklist

• Dangling pointer: a pointer still points to memory that has already been deleted.

• Memory leak: heap memory was created with new but no live pointer remains to delete it later.

• Wrong delete form: one object uses delete; arrays use delete[].

• Local copy bug: changing a pointer parameter of type int * changes only the local copy, not the caller's pointer.

Const pointer combinations

Form	What is const?
const double *p	The value pointed to should not be changed through p
double *const p = &x;	The pointer itself cannot point somewhere else
const double *const p = &x;	Neither the pointed-to value nor the pointer may change through p

```
double radius = 5;
double other = 100;

double *const p1 = &radius;   // p1 cannot point somewhere else
const double *p2 = &radius;   // *p2 should not be changed through p2
const double *const p3 = &radius;
```

Very exam-relevant pattern

If a function must change the value of a caller variable, use `int &x` or `int *x`.

If a function must change what a caller pointer points to, use `int *&ptr`.

7. Example bank - casting + time formulas

Example 1: casting snippet

```
double i = 1.7;
int defaultCast = i;
int cStyleCast = (int)i;
int staticCastInt = static_cast<int>(i);

cout << 1 / 2 << endl;                    // 0
cout << static_cast<double>(1) / 2 << endl; // 0.5
```

Example 2: metric / unit formulas

```
// mph
if (metric == 0)
{
    if (units == 'h')
        return MARATHON / bestTime;
    if (units == 'm')
        return MARATHON / (bestTime / 60.0);
}

// minutes per mile
if (metric == 1)
{
    if (units == 'h')
        return (bestTime * 60.0) / MARATHON;
    if (units == 'm')
        return bestTime / MARATHON;
}
```

What to notice

The only difference between hours and minutes is whether you divide by 60.0 or multiply by 60.0.

Use `60.0`, not `60`, to make the arithmetic clearly floating-point.

7. Example bank - header / cpp / main matching

Header

```
// raceCalc.h
#ifndef RACECALC_H
#define RACECALC_H

bool runAMarathon(double &bestTime,
                  const bool badWeather,
                  const bool trained);

#endif
```

Implementation

```
// raceCalc.cpp
#include "raceCalc.h"

bool runAMarathon(double &bestTime,
                  const bool badWeather,
                  const bool trained)
{
    if (!badWeather && trained)
    {
        bestTime -= 20;
        return true;
    }
    else if (badWeather && trained)
    {
        bestTime -= 10;
        return true;
    }
    return false;
}
```

Use this page as a prototype decoder

If the call is `runAMarathon(bestTime, ...)` and bestTime should change, reference is a natural fit.

If the call were `runAMarathon(&bestTime, ...)`, then pointer style would be more natural.

7. Example bank - arrays + file I/O

Read a file into an array

```
void ReadData(const char *filename, int &size, int myArray[])
{
    ifstream inFile;
    inFile.open(filename);

    size = 0;
    int value;
    while (inFile >> value)
    {
        myArray[size] = value;
        size++;
    }

    inFile.close();
}
```

Compute statistics from an array

```
int getLargest(int arr[], int size)
{
    int largest = arr[0];
    for (int i = 1; i < size; i++)
        if (arr[i] > largest)
            largest = arr[i];
    return largest;
}
```

Determine GPA

```
void determineGPA(const double *const list,
                  const int count, double &ave)
{
    double total = 0;
    for (int j = 0; j < count; j++)
        total += list[j];
    ave = total / count;
}
```

7. Example bank - growing a dynamic array

```
void addGPA(const double gpa, const int i, double *list)
{
    list[i] = gpa;
}

double *addNewClass(const double *const oldList,
                    const double newGpa, int &size)
{
    double *returnList = new double[size + 1];

    for (int i = 0; i < size; i++)
        returnList[i] = oldList[i];

    returnList[size] = newGpa;
    size++;
    return returnList;
}
```

Usage pattern

```
double *tempGPA = addNewClass(gpaList, addedGPA, classCount);
delete [] gpaList;
gpaList = tempGPA;
```

Why this matters

The function returns a new pointer because the old array may be too small.

The caller still owns the old heap array and must delete it after copying to the new one.

The size is passed by reference because the caller's count must increase.

7. Example bank - pointerSwap + last-minute checklist

pointerSwap

```
void pointerSwap(int *&ptr)
{
    ptr = new int;
    *ptr = 33;
}
```

Picture-style setup

```
int i = 10;
int j = 20;
int *ptrI = &i;
int *ptrJ = &j;
int *ptrArray[2];
ptrArray[0] = ptrJ;
ptrArray[1] = ptrI;
```

Last-minute checklist

Ask yourself	Why
Did I cast before division when needed?	Avoid accidental integer division.
Does the function need value, reference, pointer, or reference-to-pointer?	The interface controls whether caller data changes.
Does this pointer point to one object or to an array?	This decides `delete` vs `delete[]`.
Am I copying the call style the prompt gives me?	Important in syntax-fix / fill-in questions.
If I use file I/O, am I following the course's expected pattern?	Course-style fill-in may differ from general robust style.